

チュートリアルの進め方 — 手順に沿ってアプリを構築する

このドキュメントは、**初心者向け技術書のように「手順に沿って進めればアプリが組み上がる」**ための読み方・進め方をまとめたものです。各章の Markdown は、次の形式で実習手順を書いています。

1. 各章で使う「実習手順」の形式

各セクションには、次の4つがセットで書かれています。

項目	意味	例
手順番号	この章内で何番目の作業か	手順 9-1, 9-2, …
対象ファイル	コードを書く・編集するファイルのパス	<code>prisma/schema.prisma</code>
入力するコード (サンプルコード)	そのファイルに追加・修正する内容。写経して実行できる具体的なコードを記載	コードブロックで記載
実行方法	そのコードを動かすためのコマンドや操作 (例: 開発サーバー起動、画面の操作)	「 <code>npm run dev</code> 」してブラウザで <code>/feed</code> を開く など
実行するとこうなる	上記を実行したときに、画面・ターミナル・DB などに 実際にどう表示・出力されるか を具体的に書いたもの。初心者が「合っているか」を判断する基準になる	「画面上に『タイムライン』という見出しと投稿一覧が表示される」「ターミナルに <code>✓ Compiled</code> と出る」など
このあと変わること	そのコードを入れたことで何がかわるか	「DBに Post テーブルが定義される」など
確認方法	正しくできたかどうか確認する手順	「 <code>npx prisma generate</code> 」がエラーなく終了する など

実習ブロックの見本

各章では、次のようなブロックで「今どこに何を書くか」が分かります。

実習 9-1: Post モデルをスキーマに追加する

- ****対象ファイル****: `prisma/schema.prisma`
- ****やること****: 投稿用の `Post` モデルを定義する。
- ****入力するコード（サンプルコード）****: （ここにコードが書いてある）
- ****実行方法****: ターミナルで `npx prisma generate` を実行する。
- ****実行するところなる****: ターミナルに「Prisma Client が生成されました」のようなメッセージが表示され、
- ****このあと変わること****: Prisma が「投稿」を保存するためのテーブル定義が増える。まだ DB には反映されず、
- ****確認方法****: プロジェクトルートで `npx prisma generate` を実行し、エラーが出ないことを確認する。

2. 進め方の流れ（推奨）

1. 索引（00_index.md）で章の順番を確認する

Part 1 → Part 2 → … の順に進めると、必要な前提が揃った状態で次の章に進めます。

2. 各章の「実習手順マップ」を最初に読む

章の冒頭にある「この章で行う手順一覧」で、

- 手順番号
- 対象ファイル
- その手順を終えたときに「何ができるようになるか」を把握します。

3. 上から順に「手順」を実行する

- **対象ファイル** をエディタで開く
- **入力するコード（サンプルコード）** を写す（写経推奨）
- 保存したら **実行方法** に従って動かす
- **実行するところなる** の説明と見比べて、同じ表示・出力になっているか確認する
- **確認方法** を実行する
- **このあと変わること** を読んで、挙動の変化を理解する

4. 確認でエラーが出たら

- メッセージをよく読む

- 対象ファイルのパス・コードが写し間違っていないか確認する
- 前の手順を飛ばしていないか確認する

5. 章が終わったら

- 章末の「まとめ」「確認ポイント」で振り返る
- 必要なら `git commit` してから次章へ

3. 章別「何をどこでやるか」の一覧

索引 (00_index.md) の「**章別・実習手順サマリ**」に、章ごとに次のような表があります。

- **手順数**: その章で行う実習の個数
- **主な対象ファイル**: 編集するファイルの例
- **完了時の確認**: 章を終えたときに何をすれば「できた」と分かるか

ここを見れば、「今どの章のどのあたりをやっているか」「終わったときに何を確認すればいいか」が分かります。

4. 用語の統一

- **対象ファイル**: 編集するファイルのパス（プロジェクトルートからの相対パス）
- **入力するコード / サンプルコード**: そのファイルに書く内容。各手順で**そのまま写経して実行できる具体的なコード**を掲載する。
- **実行方法**: そのコードを動かすためのコマンドや操作（例: `npm run dev`、ブラウザで特定URLを開く、ボタンをクリックする）。
- **実行するとこうなる**: 実行したときに**画面・ターミナル・DBなどに実際にどう表示・出力されるか**を具体的に書いたもの。初心者が「自分の環境で合っているか」を判断する基準になる。
- **このあと変わる点**: その変更でアプリやプロジェクトのどこがどう変わるか（機能・画面・コマンドの変化）
- **確認方法**: 正しくできたかを確認する具体的な操作（コマンド実行、画面の確認、エラーが出ないことなど）

5. 写経について

- コードは **コピー&ペーストより、手で打つ（写経）** ことを推奨します。
- 写経すると typo でエラーが出るがありますが、そのときに「どの行が問題か」「メッセージの意味」を読む練習になります。
- 時間が足りない場合は、まず写経で一通動かし、あとから「この部分は何をしているか」を読み返す進め方でも構いません。

この「手順・対象ファイル・サンプルコード・実行方法・実行するとこうなる・変わること・確認方法」の形式に沿って、各章の Markdown を読み進めると、**どの順番でどのファイルに何を入力すればよく、実行するとどう表示・出力され、何を確認すればよいか** が分かるようになっています。

6. 各章での記載ルール（執筆・更新する方へ）

各章の実習手順では、次の項目を**省略せず**書くことを推奨します。初心者が「自分の環境で合っているか」を判断できるようにするためです。

項目	記載のコツ
入力するコード (サンプルコード)	そのまま写経して実行できる具体的なコードを掲載する。省略せず、プロジェクトで実際に使っているパス・ファイル名に合わせる。
実行方法	そのコードを動かすためのコマンド（例: <code>npm run dev</code> ）や操作（例: ブラウザで <code>/feed</code> を開く）を1~2文で書く。
実行するとこうなる	実行したときに 画面・ターミナル・DB に実際にどう表示・出力されるかを具体的に書く。 例: 「ターミナルに <code>✓ Compiled</code> と出る」「画面上に『タイムライン』という見出しと投稿一覧が表示される」。
このあと変わる こと	その変更でアプリやプロジェクトのどこがどう変わるかを簡潔に書く。
確認方法	正しくできたかを確認する具体的な操作を書く。

コード例だけでは「実行するとどうなるか」が分からないことが多いため、**実行するとこうなる**を各実習ごとに付けると、初心者には親切です。